FaithSieve: Fine-Grained Evaluation of Math Proofs with Faithful Formal Evidence

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large language models can now generate complex, multi-step mathematical proofs, but reliably determining their correctness and localizing early logical errors remains a critical challenge. Existing evaluation approaches largely depend on model-based natural-language judgments, which often overlook local reasoning gaps. While formal theorem provers like Lean offer a path to rigorous verification, using them to evaluate informal text requires solving locality and semantic mismatches: a prover might bypass a local flaw by proving an overly broad target, or validate an auto-formalized statement that drifts from the original mathematical intent. To address this, we introduce FaithSieve, a Lean-assisted framework for fine-grained evaluation of natural-language mathematical proofs. FaithSieve decomposes coarse proof steps into local reasoning units, extracts typed proof obligations, and verifies them through a formal evaluation agent. Formal validation is gated by semantic alignment scoring, so Lean evidence is incorporated only when the formal statement faithfully preserves the context, objects, and logical form of the original claim. We construct two expert-verified datasets, ProofLoc-Olympiad and ProofLoc-University, to benchmark first-error localization. On the 350-problem Olympiad dataset, FaithSieve using a GPT-5.4 backbone achieves 81.43% exact first-error accuracy, outperforming the direct-judging baseline of 72.29%. Furthermore, on the 200-problem ProofLoc-University benchmark spanning six advanced domains, FaithSieve reaches 84.5% exact accuracy, compared to 75.0% for the direct judge. Our work demonstrates that decomposing proofs into fine-grained units and grounding them with faithful formal evidence significantly improves reliable evaluation of natural-language reasoning.

1 Introduction

Large language models can now generate multi-step mathematical proofs, but evaluating them and locating the first logical error remains difficult. Such proofs may omit conditions, skip derivations, use implicit assumptions, or continue from early mistakes to plausible-looking conclusions. Reliable evaluation must therefore check not only final answers but also local reasoning steps.

Existing natural-language verification methods have made substantial progress. Self-consistency, process supervision, process reward models, and recent work on error localization, ProcessBench, Hard2Verify, and ProofGrader / ProofBench provide useful baselines for step-level evaluation [Wang et al., 2023, Lightman et al., 2023, Tyen et al., 2024, Zheng et al., 2025, Pandit et al., 2025, Ma et al., 2025]. However, their evidence mainly comes from model-based scoring or classification over natural-language reasoning.

In parallel, formal reasoning systems have also advanced rapidly. Lean, mathlib, and systems such as DeepSeek-Prover-V2, Seed-Prover, AxiomProver, Aria, and M2F show that many mathematical

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

claims can now be checked with formal evidence [The mathlib Community, 2020, Ren et al., 2025, Chen et al., 2025b,a, Axiom, 2026, Wang et al., 2025, 2026].

Using Lean as a direct judge for informal proofs still faces a central obstacle. Lean only verifies the formal statement submitted to it; even if that statement is provable, it need not faithfully express the proof step being checked. Autoformalization may omit assumptions, add conditions, reverse an implication, or rewrite a contested local claim into a different statement that is provable but not equivalent. Formal evidence is therefore useful for error localization only when the validation target remains local and semantically faithful to the original claim.

We propose FaithSieve, a Lean-assisted framework for first-error localization in natural-language mathematical proofs. FaithSieve decomposes coarse proof steps into local `EdgeUnits`, extracts typed proof obligations from suspicious transitions, and combines lightweight checking, Lean validation, and semantic alignment scoring to produce local evidence. Lean is not used as a direct judge of the whole proof; instead, it serves as a source of structured evidence gated by semantic alignment.

The overall workflow has three stages: localization, verification, and synthesis. The localization stage constructs `EdgeUnits` and selects local transitions for priority inspection. The verification stage turns these transitions into proof obligations and gathers evidence through both formal and lightweight non-formal checks. The synthesis stage aggregates local evidence back to the original step level and predicts the first erroneous step. Figure 1 illustrates this pipeline.

We construct two expert-verified datasets: PROOFLOC-OLYMPIAD, with 350 algebra and number theory problems, and PROOFLOC-UNIVERSITY, with 200 advanced mathematics problems. On PROOFLOC-OLYMPIAD, FaithSieve achieves 81.43% exact first-error accuracy, compared with 72.29% for a GPT-5.4 direct-judging baseline. On PROOFLOC-UNIVERSITY, FaithSieve also obtains clear gains over direct judging.

Our contributions are as follows.
1. **Lean-assisted first-error localization.** We propose a Lean-assisted framework that uses formal validation as structured evidence rather than as a direct judge.
2. **Obligation-centered local verification.** We introduce `EdgeUnits` and typed proof obligations to capture locally verifiable reasoning responsibilities.
3. **Semantic alignment scoring.** We develop an alignment mechanism that prevents semantically drifted Lean statements from being treated as reliable evidence.
4. **Expert-verified benchmarks.** We construct two expert-verified datasets and show that FaithSieve improves over direct-judging baselines.

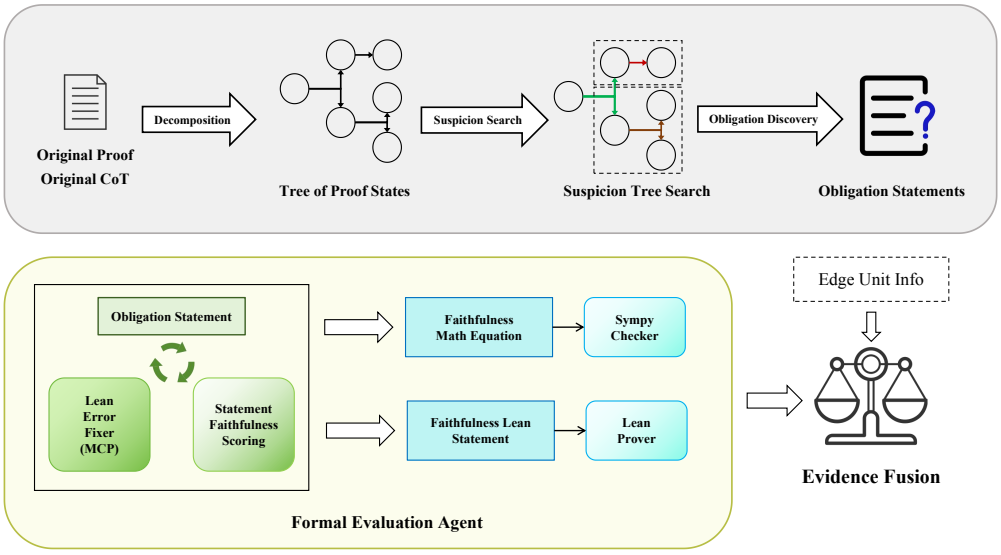

Figure 1: Overview of the FaithSieve workflow for Lean-assisted first-error localization.

2 Task Formulation and Challenges

2.1 Problem Formulation

When large language models generate natural-language mathematical proofs, a central challenge is not only judging overall correctness but also locating the first erroneous step. BIG-Bench Mistake first formulated first-mistake localization as an explicit evaluation problem, and later work extended it to mathematical process verification and open-ended solutions [Tyen et al., 2024, Zheng et al., 2025, Pandit et al., 2025]. Process supervision and PRMs provide further background for step-level correctness modeling [Lightman et al., 2023]. We adopt this formulation, but ask whether formal verification can provide checkable structured evidence for first-error localization.

Concretely, the input is a mathematical problem p and a natural-language proof $\pi = (s_1, \ldots, s_n)$ with n steps. The system must output either `correct` or the index k of the first erroneous step, where k refers to a benchmark step. We define the first erroneous step as the earliest step that makes the proof mathematically unreliable; later errors may exist, but they are not the target.

For evaluation, we use the same label interface as prior first-error localization work [Tyen et al., 2024, Zheng et al., 2025] and report binary accuracy and exact accuracy. Let $y, \hat{y} \in \{\texttt{correct}\} \cup \{1, \ldots, n\}$ denote the gold label and system prediction. Binary accuracy measures whether the system correctly distinguishes valid proofs from invalid ones, while exact accuracy requires an exact match to the gold label: `correct` for valid proofs, and the correct first-error step for invalid proofs.

2.2 Lean-Based Verification: Promise and Challenges

To go beyond purely natural-language judges, a natural direction is to use proof assistants such as Lean to provide checkable artifacts for local mathematical claims. Lean validation can turn part of the judgment from a model score into a verifiable proof result or failure signal. For algebraic rewriting, logical entailment, goal reduction, witness verification, and case coverage, formal validation may provide stronger evidence than natural-language scoring. However, this evidence is not unconditionally reliable: formal validation must target the right local object, and that object must faithfully express the original natural-language claim.

Locality mismatch. The first difficulty is ensuring that formal validation targets the same local inference as the natural-language proof. A proof usually consists of local reasoning transitions rather than a single theorem. If a proof claims to derive b from a, and then c from b, proving only $a \to c$ does not establish that the two local steps $a \to b$ and $b \to c$ are correct. A strong prover may directly prove a stronger or coarser target and ignore a local error in the original proof. Formal validation therefore requires decomposition and localization: coarse steps must be broken into local reasoning units, and the system must choose which local claims to validate.

Semantic mismatch. The second difficulty is ensuring that the Lean statement faithfully represents the original obligation. Even after localization, the system must translate the informal claim into a formal proposition. Autoformalization may omit assumptions, add conditions, reverse implications, alter quantifiers, weaken or strengthen the conclusion, or replace the claim with a different but provable statement. Thus, compilation or proof success alone is insufficient evidence; the key question is whether the Lean statement faithfully corresponds to the original claim.

These two mismatches lead to two requirements. First, formal validation should operate on local reasoning units, so the system must decompose proofs appropriately. Second, the Lean statement being validated must be semantically aligned with the original natural-language obligation. Accordingly, we use Lean not as an end-to-end theorem prover, but as a conditional evidence generator: a formal result is strong evidence only when both locality and faithfulness are satisfied.

3 Localizing Verifiable Reasoning Units

3.1 Recursive Decomposition into a Tree of Proof States

For a traditional chain-of-thought proof, a COT step is suitable as an annotation and output unit, but it is often not the smallest reasoning unit, because a single step frequently contains multiple operations simultaneously: substitution, definition expansion, introduction of new facts, goal switching,

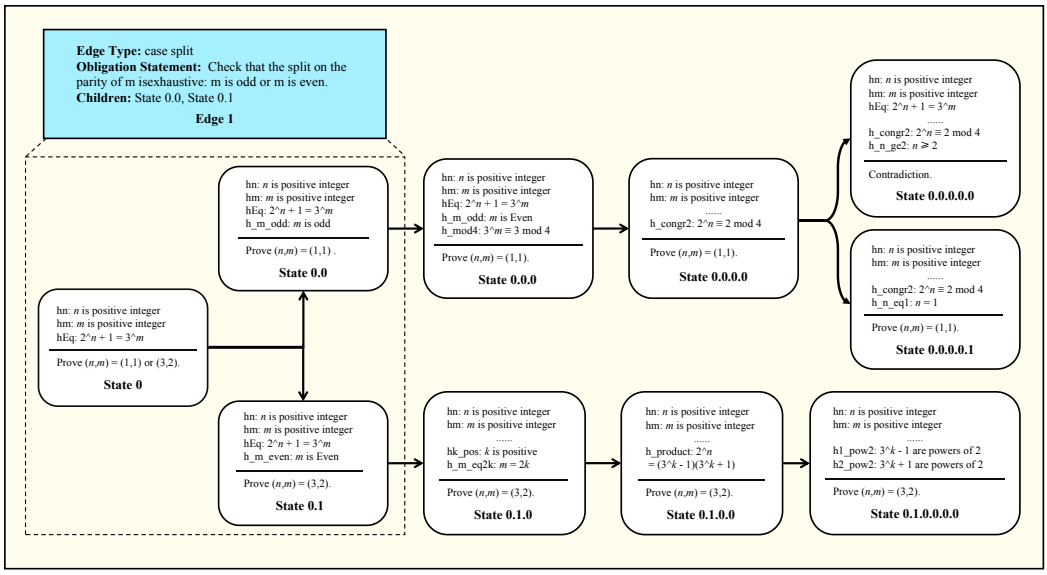

Figure 2: From Proof Steps to a Tree of Proof States

or branch pruning. If such coarse-grained steps are passed directly to a Lean pipeline, we encounter the locality mismatch discussed in Section 2. Therefore, we treat a COT step as an annotation unit, but not directly as a logical verification unit. To obtain finer-grained proofs, we first ask the language model to pre-decompose the original COT steps, removing redundant and repetitive statements without modifying the proof logic, and obtaining a detailed COT in a semantically faithful manner. Here we emphasize "faithful" rather than "patching": conclusions, variable scopes, case-discussion wording, concrete witnesses, and numerical calculations in the original proof must all be preserved as-is.

A Lean proof state records the available assumptions and current goal at a given moment, and a tactic transforms one proof state into another. Building on the fine-grained substeps obtained from preprocessing, we borrow the state-transition perspective of Lean proof states and ask the language model to write out all proof states and state transitions in the proof process step by step according to the natural-language steps, constructing a tree of proof states $\mathcal{T} = (V, E)$.

Definition of a Tree of Proof States. Each node $v \in V$ represents a natural-language proof state $S_v = (\Gamma_v, G_v)$, where Γ_v is the currently available accumulated assumptions and G_v is the current subgoal to be proved. Taking the first step in Figure 2 as an example, after the conditions and conclusion of the original problem in State 0, the natural-language proof wishes to perform a case split on the parity of m. Therefore, State 0.0 adds the branch assumption "m is odd" on top of the parent state and maintains the branch goal $(n, m) = (1, 1)$; State 0.1 adds the branch assumption "m is even" and maintains the branch goal $(n, m) = (3, 2)$. Together they correspond to the next proof states of State 0 in different proof branches. Each edge $e = (v \to v') \in E$ represents a local reasoning transition of type τ_e, such as fact derivation, rewrite, goal reduction, case split, or lemma introduction. Each edge is recorded as an `EdgeUnit` u_e, serving as the basic object for subsequent review and formal validation. In the example just given, the corresponding edge type is case split.

Recursive construction. In the process of constructing the proof state tree, for each pre-decomposed substep, the model first identifies its transition type τ_e, then locates the parent state v where this substep occurs. For different transition types, we have written a detailed state-transition manual to enable the LLM to accurately generate child states $S_{v'} = (\Gamma_{v'}, G_{v'})$. For example, if the substep introduces a new fact h, we update $\Gamma_{v'} = \Gamma_v \cup \{h\}$; if it changes the current goal, we update G_v; if it produces multiple branches, we generate multiple child nodes with branch-specific conditions. Edge 1 in Figure 2 is a case split example: it splits State 0 into State 0.0 and State 0.1, corresponding to m being odd and m being even, respectively. We further list state-update templates for different logical patterns in the appendix. Thus, recursive construction is not merely text segmentation, but continuously maintains the local proof context carried by each `EdgeUnit`.

3.2 Suspicion Tree Search

After proof-step pre-decomposition and recursive construction of the proof-state tree, we obtain a collection of `EdgeUnits` that is often much larger than the original set of COT steps. Since Lean validation is expensive, it is impractical to generate formal validation results for every `EdgeUnit`. We therefore first run Suspicion Search over all edges in the proof-state tree, producing an initial risk tree that estimates which local transitions are more likely to contain the first material error.

Suspicion Search uses the proof-state information attached to each `EdgeUnit`, including before-context, after-context, goal relation, branch scope, and transition type. The model produces a set of one-shot lightweight judgments: whether the transition is suspicious, how suspicious it is, and an overall suspicion risk score in $[0, 1]$. During the search, we provide examples of common failure modes for different edge types and prompt the model to check for the corresponding logical errors. Empirically, more than 80% of suspicion risk scores fall into the two extreme intervals $[0, 0.2]$ and $[0.8, 1.0]$. We therefore treat edges with suspicion risk above 0.6 as relatively suspicious candidates.

After scoring all edges, we build an audit window rather than selecting the globally most suspicious `EdgeUnit`. Among suspicious edges, we choose the earliest one as the focus unit and include a short prefix of preceding edges, especially those with special transition types. This yields a cost-controlled set of local transitions for obligation discovery and validation.

3.3 Obligation Statement Generation

Proof-state tree decomposition gives us local transitions, but a transition itself is not yet a directly checkable mathematical statement. An `EdgeUnit` records how the proof moves from a before-state to an after-state: available assumptions, newly introduced conditions, goal changes, and branch or source-step scope. To validate such a transition, we summarize its mathematical responsibility as an explicit obligation statement. In most cases, this statement has an implication-like form: under the before-context, a new claim follows; or, under the incoming context, proving the after-goal is sufficient for proving the before-goal.

Concretely, we generate obligations by selecting a conversion template according to the `EdgeUnit`'s edge type and state change. For common logical actions, we provide examples that map a before-state–after-state transition to an obligation statement, and ask the language model to instantiate the corresponding template for the current edge. The model therefore does not freely summarize the whole proof; it generates a statement matched to the local transition under the given proof states, transition type, new conditions, and goal relation. Table 1 summarizes the main obligation kinds and their corresponding local checks.

For example, in a `fact_plus_reduction` transition, an edge both introduces a new fact and changes the goal. If we only check whether the after-goal is sufficient for the before-goal, the new claim may be used as a premise before being validated. We therefore split this transition into two obligations: whether the new fact is supported by the prior context, and whether the goal reduction is valid when that fact is available. For a `case_split`, the obligation instead checks whether the cases are exhaustive, form a valid partition, or justify the stated branch consequence. Thus, typed conversion turns a local edge into an explicit statement that formal validation can handle.

Table 1: Typed obligation kinds for selected `EdgeUnits`.

Type	Local check	Trigger
Derived fact	New fact follows from context	Implication to new claim
Rewrite	Rewrite or calculation is valid	Equivalent rewrite or simplification
Goal reduction	After-goal implies before-goal	Sufficient condition for goal
Fact plus reduction	New fact and reduction are valid	Claim + goal change
Case split	Cases are valid and covering	Case analysis
Branch elimination	Discarded branch is impossible	Infeasible branch
Witness	Witness satisfies the target	Witness term
Contested claim	Flagged claim is true	Flagged claim
Entailment	Context entails the claim	Bare assertion
Overlap partition	Cases form a proper partition	Partition claim

4 Faithfulness-Aware Verification and Evidence Fusion

4.1 Statement Faithfulness Scoring

Given an obligation o, let m denote its natural-language statement and C its local context. Formal validation first generates a Lean statement $\hat{m}$. Since Lean validates only $\hat{m}$, if $\hat{m}$ drops assumptions, changes objects, reverses the implication direction, or weakens the original claim, then a successful Lean proof does not establish the original obligation. Formal validation must therefore be preceded by statement faithfulness checking.

To measure the semantic faithfulness of the formal statement $\hat{m}$ to the natural-language obligation m under the local context C, we define the statement faithfulness score $S_{\text{faith}}(m, \hat{m}; C) \in [0, 1]$:

$$S_{\text{faith}}(m, \hat{m}; C) = \sqrt{S_{\text{prem}}(m, \hat{m}; C) \cdot S_{\text{conc}}(m, \hat{m}; C)} \cdot S_{\text{hol}}(m, \hat{m}; C). \tag{1}$$

Equation (1) combines premise fidelity, conclusion fidelity, and a holistic semantic score to measure whether $\hat{m}$ still expresses the same local mathematical claim. Here, S_{prem} measures whether the hypotheses of $\hat{m}$ faithfully retain the local context, branch assumptions, and fixed witnesses needed by the obligation, while S_{conc} measures whether the conclusion of $\hat{m}$ matches the target claim of m. Let $T_{\text{prem}}(m, C)$ denote the premise-side semantic slots extracted from the context and the source step, and let $T_{\text{conc}}(m)$ denote the conclusion-side semantic slots in the obligation. The value $\text{match}(t, \hat{m}) \in [0, 1]$ is assigned by the semantic checker rather than by string matching: for each semantic slot t, the checker compares the source slot with the hypotheses or conclusion of $\hat{m}$, and judges whether it is preserved, omitted, weakened, strengthened, or replaced. Preserved slots receive high scores, while missing critical assumptions, altered objects, or changed claims receive low scores. The resulting slot scores are defined in Equation (2).

$$S_{\text{prem}} = \frac{\sum_{t \in T_{\text{prem}}} \text{match}(t, \hat{m})}{|T_{\text{prem}}|} \quad S_{\text{conc}} = \frac{\sum_{t \in T_{\text{conc}}} \text{match}(t, \hat{m})}{|T_{\text{conc}}|}. \tag{2}$$

The holistic score S_{hol} is determined by five aspects: step-relation fidelity, object and witness fidelity, directionality fidelity, role-alignment fidelity, and syntax fidelity. It captures severe patterns beyond isolated premise or conclusion slots, including wrong direction, role reversal, meta-level wrappers, and vacuous formalizations; details are in the Appendix.

Together with the faithfulness score, the semantic checker returns a structured semantic report. This report summarizes the diagnostic information produced when the checker evaluates different aspects and semantic elements, including statement-level status, detected drift categories, checker confidence, a natural-language reason, missing assumptions, overgeneralization examples, and a suggested revision. It points out the semantic differences between m and $\hat{m}$: which contextual assumptions are missing, which mathematical objects have been replaced, which implication direction or quantifier structure has changed, and what should be added or revised to repair $\hat{m}$.

4.2 Formal Evaluation Agent

Using Statement Faithfulness Scoring as a central checking tool, we develop the Formal Evaluation Agent to convert each typed obligation into local checkable evidence. The agent runs on each obligation and returns a local evaluation result containing: (i) the obligation's local checking status in $\{\texttt{passed}, \texttt{refuted}, \texttt{inconclusive}\}$, (ii) evidence strength adjusted by faithfulness and checker confidence, and (iii) a checkable artifact or diagnostic message. This result answers only the local obligation; it is not a proof-level verdict.

Formal Statement Generation. First, the statement agent generates a local formal statement $\hat{m}$ or a LaTeX mathematical equation from the problem statement, the `EdgeUnit` summary, and the obligation statement. The latter is generated only when the obligation is a pure mathematical expression. When generating $\hat{m}$, the system first asks the LLM for candidate statements and then checks whether the candidate compiles. If it does not compile, the system invokes Lean MCP tools to repair syntax, type, or dependency errors. After compilation succeeds, the semantic checker performs faithfulness scoring between $\hat{m}$ and the original obligation m, returning S_{faith}, component-level semantic judgments, and a structured semantic report. If the language-meaning score or a critical component score falls below the threshold, the agent revises the statement using the drift report and repeats both compilation and semantic checking.

Equation SymPy Checker. We quickly handle pure numeric claims extracted in the previous step, provided they contain neither variables nor natural-language explanation. Concretely, the system normalizes mathematical notation from the LaTeX equation and passes the resulting expression to a SymPy-based numeric checker. This checker handles only pure numeric equality chains and single comparison statements, returning `passed`, `refuted`, or `inconclusive`.

Lean Prover Checker. For other obligations whose statements pass the faithfulness gate, the agent enters the Lean validation branch. According to the suspicion score from the audit stage, the system verifies the original proposition in Lean for obligations that are likely correct, and verifies a negated proposition or the existence of a counterexample for obligations that are likely wrong. A proof-search agent repeatedly interacts with the Lean MCP tools and revises the proof attempt until it succeeds or reaches the time limit. If the original proposition is proved, the result is `passed`; if the negated proposition or a counterexample is verified, the result is `refuted`. Timeout or failure to pass the faithfulness gate produces `inconclusive`, together with a specific inconclusive type.

Evidence Fusion and Final Decision. After obtaining the local checking results from the Formal Evaluation Agent, the system aggregates all evidence and makes the final decision. Since one `EdgeUnit` may produce multiple obligations, fusion first summarizes each u_e's local review, typed obligations, formal evaluation results, faithfulness reports, proof diagnostics, and inconclusive reasons into unit-level evidence $\mathrm{Info}(u_e)$, and maps it back to the original COT step. The system then merges all relevant $\mathrm{Info}(u_e)$ at the step level: if there exists faithful and reliable negative evidence, such as a refuted obligation, a verified counterexample, or agreement between formal diagnostics and local review on a logical error, the step is marked `incorrect`; if the main evidence comes from semantic drift, formalization failure, context inconsistency, timeout, or conflicting diagnostics, it is marked `uncertain`; only when the related local evidence supports the step and no unresolved risk remains is it marked `correct`. Finally, the system selects the earliest step with reliable negative evidence as the first error and checks the preceding uncertain prefix for an earlier root-cause candidate; if no reliable negative evidence is found, the LLM synthesizes all local evidence and outputs either `correct` or a risky step. The complete procedure is shown in Algorithm 1.

Algorithm 1: Lean-assisted first-error localization

Input: Problem p; natural-language proof $\pi = (s_1, \ldots, s_n)$
Output: $\hat{y} \in \{\texttt{correct}\} \cup \{1, \ldots, n\}$

```
Decompose each coarse step s_i into faithful fine-grained substeps
Build state tree T = (V, E) and normalize edges into mapped EdgeUnits
Scan EdgeUnits and schedule audit window W by earliest risk and prefix coverage
foreach scheduled EdgeUnit u_e ∈ W do
    Run local natural-language review and compile typed obligations O(u_e)
    foreach obligation o ∈ O(u_e) do
        Generate a Lean statement m̂ or a LaTeX equation from p, u_e, o
        if o is extracted as a pure numeric claim then
            Normalize the equation and run the SymPy numeric checker
        else
            Compile m̂, repairing syntax, type, or dependency errors if needed
            Score faithfulness and return semantic judgments and a structured report
            if language-meaning or critical component score is low then
                Revise m̂ using the drift report and repeat checking
            if m̂ still fails the faithfulness gate or checking times out then
                Return inconclusive with a specific type
            else
                Run Lean prover checker on the claim, negation, or counterexample branch
        Return local result: passed, refuted, or inconclusive
    Aggregate review, obligations, formal results, reports, and diagnostics into Info(u_e)
Map Info(u_e) back to original coarse COT steps
if faithful reliable negative evidence is found then
    Check the preceding uncertain prefix and return the first-error step
else
    return correct or a risky step after final LLM synthesis
```

Table 2: PROOFLOC-UNIVERSITY domain distribution.

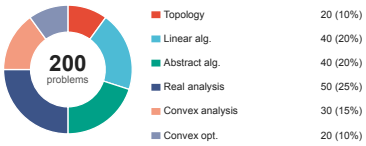

Table 3: Direct-judge base-model results.

Base model	Params.	Exact (%)	Binary (%)
GPT-5.4 Direct	undisclosed	72.29	88.29
Gemini-3.1-Pro Direct	undisclosed	70.86	82.86
Opus-4.6 Direct	undisclosed	71.71	89.14
DeepSeek-v4-Pro Direct	undisclosed	48.00	71.14
GLM-5.1 Direct	undisclosed	57.71	78.57
Kimi-k2.6 Direct	undisclosed	63.43	80.86
Qwen3.5-9B Direct	9B	53.71	76.00

5 Empirical Evaluation

This section evaluates our method on first-error localization for natural-language mathematical proofs. We first introduce two expert-verified datasets, then compare direct-judging performance across several base models, and finally report the performance of the full FAITHSIEVE system together with component ablations. Unless otherwise specified, ablations are conducted on PROOFLOC-OLYMPIAD; unreported PROOFLOC-UNIVERSITY backbone entries are marked as TBD.

5.1 Datasets

We construct two expert-verified datasets targeting olympiad-style proofs and university-level textbook proofs. The first dataset, PROOFLOC-OLYMPIAD, contains 350 algebra and number theory problems. The second dataset, PROOFLOC-UNIVERSITY, contains 200 problems across topology, linear algebra, abstract algebra, real analysis, convex analysis, and convex optimization. Candidate proofs in both datasets are generated by GPT-4o and then annotated by experts for proof correctness and the first erroneous coarse proof step. Importantly, the ground-truth label is aligned with the benchmark-level proof step, rather than the internal `EdgeUnit` used by our system.

5.2 Baselines and System Performance

We first evaluate the direct-judge baseline. This baseline provides the model with the problem statement and the original natural-language proof, and asks it to output either `correct` or the first erroneous step in one pass. It does not use the tree of proof states, `EdgeUnits`, local review, typed obligations, the Formal Evaluation Agent, or Evidence Fusion. Thus, direct judging measures the ability of base models to localize the first error without structured decomposition or formal evidence.

The base models include GPT-5.4, Gemini-3.1-Pro, Opus-4.6, DeepSeek-v4-Pro, GLM-5.1, Kimi-k2.6, and Qwen3.5-9B, covering recent frontier closed models and one 9B-scale open-weight model. Model information follows the corresponding provider model cards, technical reports, or official documentation [OpenAI, 2026, Google DeepMind, 2026, Anthropic, 2026, DeepSeek-AI, 2026, Zhipu AI, 2026, Moonshot AI, 2026, Qwen Team, 2026].

GPT-5.4 direct achieves the strongest exact accuracy among the listed models, while Opus-4.6 obtains the strongest binary accuracy. Other strong models show substantial variation, suggesting that the task is not determined solely by model scale or final-answer ability. Qwen3.5-9B provides a useful 9B-scale comparison point for structured verification. These direct-judge results show that even strong models remain vulnerable to final-answer bias, local reasoning gaps, later salient errors, and benchmark-aligned materiality. This motivates our use of local `EdgeUnits`, typed obligations, and faithfulness-aware formal evidence.

Table 4 reports the performance of FAITHSIEVE under different backbones and compares it with the corresponding direct-judge setting. FAITHSIEVE denotes the full pipeline in Sections 3 and 4: trees of proof states and `EdgeUnits`, suspicion-guided audit windows, typed obligation discovery, the Formal Evaluation Agent, statement faithfulness scoring, and Evidence Fusion.

With the GPT-5.4 backbone, FAITHSIEVE achieves 81.43% exact accuracy and 93.71% binary accuracy on PROOFLOC-OLYMPIAD, outperforming GPT-5.4 direct judging at 72.29% and 88.29%. On PROOFLOC-UNIVERSITY, FAITHSIEVE reaches 88.50% exact / 93.50% binary accuracy, compared

Table 4: Main FAITHSIEVE results.

Dataset	Backbone / setting	Method	Exact (%)	Binary (%)
PROOFLOC-OLYMPIAD	GPT-5.4	Direct judge	72.29	88.29
PROOFLOC-OLYMPIAD	GPT-5.4	FAITHSIEVE	81.43	93.71
PROOFLOC-OLYMPIAD	Qwen3.5-9B	Direct judge	53.71	76.00
PROOFLOC-OLYMPIAD	Qwen3.5-9B	FAITHSIEVE	61.14	82.29
PROOFLOC-UNIVERSITY	GPT-5.4	Direct judge	75.50	85.00
PROOFLOC-UNIVERSITY	GPT-5.4	FAITHSIEVE	88.50	93.50
PROOFLOC-UNIVERSITY	Qwen3.5-9B	Direct judge	TBD	TBD
PROOFLOC-UNIVERSITY	Qwen3.5-9B	FAITHSIEVE	TBD	TBD

Table 5: PROOFLOC-UNIVERSITY domain results.

Domain	#Problems	FAITHSIEVE		GPT-5.4 direct	
		Exact (%)	Binary (%)	Exact (%)	Binary (%)
Overall	200	88.50	93.50	75.50	85.00
Topology	20	80.00	90.00	75.00	75.00
Linear algebra	40	97.50	97.50	92.50	92.50
Abstract algebra	40	92.50	92.50	87.50	90.00
Real analysis	50	92.00	98.00	76.00	90.00
Convex analysis	30	80.00	93.33	40.00	66.67
Convex optimization	20	75.00	80.00	70.00	85.00

with 75.50% exact / 85.00% binary for GPT-5.4 direct judging. With the Qwen3.5-9B backbone on PROOFLOC-OLYMPIAD, FAITHSIEVE improves over direct judging from 53.71% exact / 76.00% binary to 61.14% exact / 82.29% binary.

Table 5 further reports the domain-level PROOFLOC-UNIVERSITY results. The largest exact-accuracy gains appear in Real Analysis and Convex Analysis, where local formal obligations help expose errors that direct judging often misses.

5.3 Ablation Study

We further conduct component ablations on PROOFLOC-OLYMPIAD with two backbones, GPT-5.4 and Qwen3.5-9B, to analyze which designs are most important for first-error localization in FAITHSIEVE. The GPT-5.4 ablation compares the full system with variants that remove individual components. The Qwen3.5-9B ablation uses a replay-style setting, where existing intermediate artifacts are fixed and downstream components are replayed, so that the effect of each component can be compared more directly. Table 6 reports the ablation results for the two backbones.

The Step3 graph judge keeps proof decomposition and proof-state graph artifacts, but does not run the full local review, Lean validation, or evidence fusion. This experiment mainly tests the tree decomposition hypothesis: whether the tree of proof states and graph-level organization alone can produce a clear improvement without Lean evidence. With GPT-5.4, this setting reaches 75.71% exact / 88.57% binary. Compared with Direct judge, exact accuracy improves more substantially than binary accuracy, suggesting that graph decomposition mainly improves precise erroneous-step localization rather than the coarse correct/incorrect decision. The result also shows that graph structure helps organize the proof process, but graph information alone cannot replace obligation-level evidence.

The EdgeUnit + NL only experiment keeps EdgeUnit-level local review and final synthesis, but disables the cheap checker and Lean formalization. With GPT-5.4, this setting obtains 73.43% exact / 88.00% binary; with Qwen3.5-9B, EdgeUnit + NL only obtains 71.14% exact / 86.00% binary. The gain over Direct judge is especially clear for Qwen3.5-9B, indicating that local EdgeUnit representations help a smaller model focus on checkable reasoning transitions. The GPT-5.4 result still shows that formal or semi-formal evidence is needed.

The locality ablation removes local EdgeUnits and falls back to coarse-step-level judging. GPT-5.4 with w/o local EdgeUnits drops to 60.86% exact / 72.29% binary, and Qwen3.5-9B with w/o local

Table 6: Ablation results on PROOFLOC-OLYMPIAD.

GPT-5.4			Qwen3.5-9B		
Experiment	Exact (%)	Binary (%)	Experiment	Exact (%)	Binary (%)
Full FAITHSIEVE	81.43	93.71	Full FAITHSIEVE	74.00	89.14
Direct judge	72.29	88.29	Direct judge	60.00	74.57
Step3 graph judge	75.71	88.57	Step3 graph judge	65.71	77.71
EdgeUnit + NL only	73.43	88.00	EdgeUnit + NL only	71.14	86.00
w/o local EdgeUnits	60.86	72.29	w/o local EdgeUnits	62.29	77.71
w/o semantic gate	70.86	86.29	w/o semantic gate	70.86	86.00
w/o evidence fusion	66.29	82.57	w/o evidence fusion	66.00	82.57

EdgeUnits obtains 62.29% exact / 77.71% binary. This experiment directly supports the locality hypothesis: a benchmark step is an annotation unit, but not necessarily the actual reasoning unit. Without decomposing coarse proof steps into local transitions, the system loses key evidence needed for first-error localization.

The semantic-gate ablation relaxes the translation-confidence and statement-confidence thresholds, allowing hybrid-validation results to enter the final evidence set more easily. GPT-5.4 with w/o semantic gate drops to 70.86% exact / 86.29% binary; Qwen3.5-9B with w/o semantic gate obtains 70.86% exact / 86.00% binary, also below the corresponding Full FAITHSIEVE result. These results show that a Lean or checker success signal is reliable only when the statement remains semantically aligned with the original obligation; without faithfulness filtering, semantic drift contaminates the final judgment.

The evidence-fusion ablation replaces the full fusion module with a simplified rule. GPT-5.4 with w/o evidence fusion obtains 66.29% exact / 82.57% binary, and Qwen3.5-9B with w/o evidence fusion obtains 66.00% exact / 82.57% binary. This shows that obligation-level evidence cannot be mapped to the final step label by a single simple rule. The system needs to combine evidence strength, semantic faithfulness, locality, error materiality, and earliest-step consistency to produce stable benchmark-aligned first-error predictions.

6 Limitations and Conclusion

6.1 Limitations

Although our method improves exact and binary accuracy for first-error localization on PROOFLOC-OLYMPIAD, it is limited by the cost and coverage of formal validation. Lean-assisted verification involves statement generation, compile repair, proof search, artifact checking, and evidence normalization; any stage may fail or time out. The method also depends on current LLM agents for autoformalization and proof repair, so unfamiliar Lean representations or mismatches between mathlib definitions and natural-language conventions can turn an otherwise checkable claim into inconclusive evidence.

A second limitation is domain coverage. PROOFLOC-OLYMPIAD focuses on algebra and number theory, where local transformations often form clear typed obligations. Geometry and combinatorics require diagrammatic objects, construction relations, counting models, and problem-specific encodings, so failures may reflect modeling choices or library coverage. PROOFLOC-UNIVERSITY expands the evaluation to six abstract domains, but broader university-level mathematics still requires additional annotation and verification.

6.2 Conclusion

This paper studies how Lean validation can serve as structured evidence for correctness judgment and first-error localization in natural-language mathematical proofs. Our central view is that formal validation can complement natural-language judging, but only when the target is local and the formal statement faithfully represents the original obligation. Based on this view, FAITHSIEVE decomposes benchmark-level proof steps into `EdgeUnits`, selects suspicious local transitions, extracts typed

obligations, combines cheap checking with Lean validation and semantic alignment, and then fuses obligation-level evidence into a benchmark-aligned prediction.

Empirical results show that local and semantically faithful formal evidence improves first-error localization. On PROOFLOC-OLYMPIAD, FAITHSIEVE with GPT-5.4 reaches 81.43% exact accuracy and 93.71% binary accuracy, outperforming GPT-5.4 direct judging at 72.29% and 88.29%. On PROOFLOC-UNIVERSITY, it reaches 88.50% exact / 93.50% binary accuracy, compared with 75.50% / 85.00% for GPT-5.4 direct judging. With Qwen3.5-9B on PROOFLOC-OLYMPIAD, it improves over direct judging from 53.71% exact / 76.00% binary to 61.14% exact / 82.29% binary.

Future work includes expanding Lean/mathlib coverage and domain-specific encodings for geometry, combinatorics, and modeling-heavy problems; improving statement generation, semantic checking, and repair; reducing validation cost so larger audit windows can be used; and extending expert-verified datasets to broader abstract and definition-heavy mathematics.

References

Anthropic. Claude Opus 4.6 model documentation, 2026. Provider model documentation.

Axiom. AXLE documentation, 2026. URL `https://axle.axiommath.ai/v1/docs/`. Accessed: 2026-05-02.

Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, Zhihong Wang, Mingxuan Wang, Chenrui Wei, Shufa Wei, Huajian Xin, Fan Yang, Weihao Gao, Zheng Yuan, Tianyang Zhan, Zeyu Zheng, Tianxi Zhou, and Thomas Hanwen Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025a. doi: 10.48550/arXiv.2512.17260. URL `https://arxiv.org/abs/2512.17260`.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025b. doi: 10.48550/arXiv.2507.23726. URL `https://arxiv.org/abs/2507.23726`.

DeepSeek-AI. DeepSeek-v4-Pro technical report or model documentation, 2026. Provider model documentation.

Google DeepMind. Gemini 3.1 Pro model documentation, 2026. Provider model documentation.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. doi: 10.48550/arXiv.2305.20050. URL `https://arxiv.org/abs/2305.20050`.

Wenjie Ma, Andrei Cojocaru, Neel Kolhe, Bradley Louie, Robin Said Sharif, Haihan Zhang, Vincent Zhuang, Matei Zaharia, and Sewon Min. Reliable fine-grained evaluation of natural language math proofs. *arXiv preprint arXiv:2510.13888*, 2025. doi: 10.48550/arXiv.2510.13888. URL `https://arxiv.org/abs/2510.13888`.

Moonshot AI. Kimi-k2.6 model documentation, 2026. Provider model documentation.

OpenAI. GPT-5.4 model documentation, 2026. Provider model documentation.

Shrey Pandit, Austin Xu, Xuan-Phi Nguyen, Yifei Ming, Caiming Xiong, and Shafiq Joty. Hard2verify: A step-level verification benchmark for open-ended frontier math. *arXiv preprint arXiv:2510.13744*, 2025. doi: 10.48550/arXiv.2510.13744. URL `https://arxiv.org/abs/2510.13744`.

Qwen Team. Qwen3.5-9B technical report or model documentation, 2026. Provider model documentation.

Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025. doi: 10.48550/arXiv.2504.21801. URL https://arxiv.org/abs/2504.21801.

The mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. Association for Computing Machinery, 2020. doi: 10.1145/3372885.3373824. URL https://dl.acm.org/doi/10.1145/3372885.3373824.

Gladys Tyen, Hassan Mansoor, Victor Carbune, Peter Chen, and Tony Mak. LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13894–13908, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.826. URL https://aclanthology.org/2024.findings-acl.826/.

Hanyu Wang, Ruohan Xie, Yutong Wang, Guoxiong Gao, Xintao Yu, and Bin Dong. Aria: An agent for retrieval and iterative auto-formalization via dependency graph. *arXiv preprint arXiv:2510.04520*, 2025. doi: 10.48550/arXiv.2510.04520. URL https://arxiv.org/abs/2510.04520.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2203.11171. URL https://arxiv.org/abs/2203.11171.

Zichen Wang, Wanli Ma, Zhenyu Ming, Gong Zhang, Kun Yuan, and Zaiwen Wen. M2F: Automated formalization of mathematical literature at scale. *arXiv preprint arXiv:2602.17016*, 2026. doi: 10.48550/arXiv.2602.17016. URL https://arxiv.org/abs/2602.17016.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Processbench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. doi: 10.48550/arXiv.2412.06559. URL https://arxiv.org/abs/2412.06559. arXiv:2412.06559.

Zhipu AI. GLM-5.1 model documentation, 2026. Provider model documentation.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Additional Details

Put proofs, extra experiments, ablations, dataset details, and implementation details here.